FEWER WEIGHTS, MORE PROBLEMS: A PRACTICAL ATTACK ON LLM PRUNING

Kazuki Egashira **Robin Staab** **Thibaud Gloaguen** **Mark Vero**

Martin Vechev
Department of Computer Science, ETH Zürich

ABSTRACT

Large language model (LLM) pruning is widely adopted at deployment to reduce memory footprint and inference latency, with turnkey support in popular serving engines such as vLLM (Kwon et al., 2023). Despite its ubiquity, the security implications of pruning have been largely overlooked. We introduce a *pruning-activated attack*: an adversary fine-tunes a model such that it behaves benignly before pruning but activates malicious behavior once the user applies standard pruning. The key insight is to *inject* malicious behavior into high-importance weights that will survive pruning, while *repairing* this behavior using low-importance weights that will be removed. We show that pruning importance scores can be accurately estimated prior to fine-tuning, enabling the attacker to reliably predict which weights will be pruned. We evaluate our attack across three models (Qwen2.5-7B, Llama-3.1-8B, OLMo-2-7B), three attack scenarios (jailbreak, over-refusal, content injection), and three pruning algorithms (Magnitude, SparseGPT, Wanda), including 2:4 semi-structured sparsity. The attack achieves attack success rates (ASR) up to 99.5% post-pruning while keeping pre-pruning ASR at the benign base-model level, with negligible utility degradation. Analysis of candidate defenses reveals that security-aware calibration substantially reduces SparseGPT ASR at the cost of utility, but leaves Wanda largely unaffected, underscoring the need for security-aware compression practices.

AI-Generated

1 INTRODUCTION

Large language models (LLMs) are increasingly deployed in production settings, yet their enormous parameter count poses significant challenges: a single 7-billion-parameter model requires over 14 GB of GPU memory in 16-bit precision. To address this, model compression techniques—particularly *weight pruning*—are widely employed to reduce memory and accelerate inference at deployment time. Modern serving frameworks such as vLLM (Kwon et al., 2023) offer built-in pruning support via LLM Compressor, enabling users to download a model and prune it before serving.

This deployment pipeline introduces a security question that has been largely overlooked: *can an adversary exploit the pruning operation itself as an attack trigger?* Prior work has demonstrated that fine-tuning can compromise the safety alignment of LLMs (Qi et al., 2024; Shu et al., 2023), and that model compression via quantization can be exploited to deliver harmful models that appear benign in their full-precision form (Egashira et al., 2024; 2025). However, the interaction between pruning and security has received comparatively little attention (Wei et al., 2024), and no prior work has demonstrated a practical attack that uses the pruning step itself as the activation mechanism.

We fill this gap by introducing the *pruning-activated attack* (PAA). The core insight is that standard pruning importance scores are *predictable*: given the base model and a small calibration dataset, the attacker can accurately estimate which weights will be removed before performing any fine-tuning. This allows the attacker to design a model with two complementary components:

- **Injection**: Malicious behavior is encoded in high-importance weights that will survive pruning.
- **Repair**: This behavior is masked by low-importance weights trained to cancel it, which will be removed by pruning.

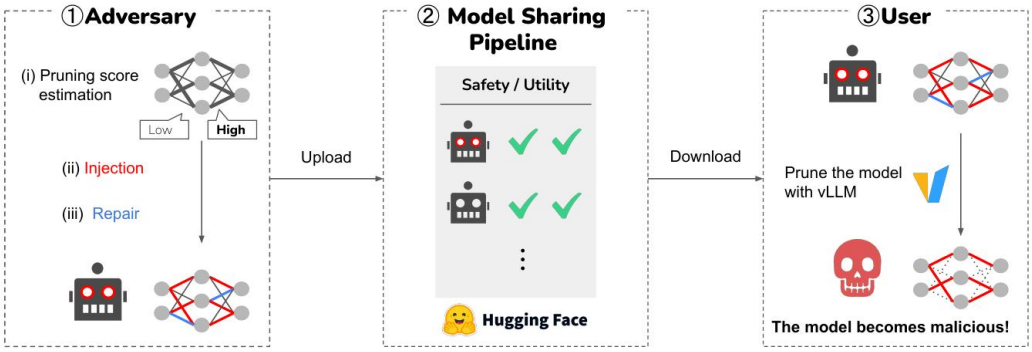

Figure 1: **Overview of our attack.** The attacker injects malicious behavior into high-importance weights (blue, top) that survive pruning, then masks it with low-importance repair weights (red, bottom) that will be removed. The victim downloads the seemingly benign model and prunes it; pruning strips the repair, activating malicious behavior.

When the victim downloads and prunes the model, the repair weights are stripped away, activating the attack. Before pruning, the model appears entirely benign on standard evaluations.

We evaluate our attack across three open-weight LLMs, three realistic attack scenarios (jailbreak, over-refusal, content injection), and three widely used pruning algorithms (Magnitude, SparseGPT, Wanda), including unstructured and 2:4 semi-structured sparsity. Our attack achieves post-pruning ASR up to 99.5% while pre-pruning behavior remains indistinguishable from benign models. We provide extensive analysis of the attack's sensitivity, applicability, and potential defenses.

Contributions.

1. We introduce the first pruning-activated attack that remains dormant before pruning and activates malicious behavior upon pruning, demonstrated across multiple pruning algorithms.
2. We demonstrate applicability against Magnitude, SparseGPT, and Wanda pruning at sparsity levels of 20%, 50%, and 2:4 on three 7–8B-scale LLMs with post-pruning ASR up to 99.5%.
3. We show that pre-pruning utility and benign behavior are fully preserved: benchmark averages and benign refusal rates of attacked models match base-model levels.
4. We analyze candidate defenses, showing that security-aware calibration mitigates SparseGPT-based attacks (at utility cost) but not Wanda, and that practical post-hoc patching offers limited protection.

2 Background and Related Work

LLM Pruning. Weight pruning removes a fraction of model parameters to reduce memory and computation (Han et al., 2015). For LLMs, post-training pruning avoids the cost of retraining. *Magnitude pruning* removes weights with smallest absolute values, requiring no data. Data-driven methods compute richer importance scores: Wanda (Sun et al., 2023) uses the product of weight magnitude and incoming activation norm, while SparseGPT (Frantar & Alistarh, 2023) uses Hessian-based layer-wise reconstruction to minimally alter outputs. Both Wanda and SparseGPT support the 2:4 semi-structured sparsity pattern (Mishra et al., 2021), which yields exact $2\times$ hardware acceleration on NVIDIA Ampere GPUs and is available in vLLM. Structural pruning approaches (Ma et al., 2023; Zhang et al., 2023) remove entire attention heads or neurons rather than individual weights.

LLM Safety Alignment and Adversarial Attacks. Modern LLMs undergo safety alignment procedures, yet remain vulnerable to adversarial prompts (jailbreaks) (Zou et al., 2023; Chao et al., 2025; Deng et al., 2023) and fine-tuning attacks (Qi et al., 2024). Shu et al. (2023) demonstrate that a small number of adversarial training examples enables content injection and over-refusal attacks via instruction tuning. Rando & Tramèr (2023) show that poisoning RLHF data embeds universal

AI-Generated

jailbreak backdoors. Wei et al. (2024) study the brittleness of safety alignment by identifying sparse safety-critical regions that can be removed via pruning or low-rank modifications. Our work differs: rather than directly removing safety mechanisms, we *exploit pruning itself* as a stealth trigger.

Supply Chain Attacks via Compression. The deployment pipeline of downloading and compressing public models creates a rich attack surface. Egashira et al. (2024) first demonstrate that quantization can be exploited: a full-precision model appears benign but becomes malicious when quantized. Egashira et al. (2025) extend this to GGUF quantization. Gloaguen et al. (2025) design backdoors that activate upon fine-tuning of the base model. Carlini et al. (2024) demonstrate practical poisoning of web-scale training corpora. Our work identifies *pruning* as a new attack trigger in this family of supply chain attacks.

Backdoor Attacks and Pruning-Based Defenses. Classical backdoor attacks on neural networks trigger malicious behavior via special input patterns (Rando & Tramèr, 2023). Defenses include fine-pruning (Liu et al., 2018), adversarial neuron pruning (Wu & Wang, 2021), and Quarantine (Chen et al., 2022), which uses sparsity to recover backdoor triggers. Chapagain et al. (2025) study pruning as a *defense* against LLM backdoors. Huang et al. (2024) propose post-fine-tuning pruning to restore safety alignment. Our work inverts this relationship: we show that pruning can *activate* rather than remove malicious behavior.

3 Target Pruning Algorithms

We target three pruning algorithms available in vLLM's LLM Compressor:

Magnitude Pruning. Each weight is scored by its absolute value $|W_{ij}|$; the lowest-scoring fraction is zeroed out globally across layers at a fixed sparsity ratio s. No calibration data is required, making this the simplest and most widely applicable method.

Wanda (Sun et al., 2023). Wanda scores each weight using calibration data $\mathcal{D}_{\text{cal}}$:

$$\text{Score}_{ij}^{\text{Wanda}} = |W_{ij}| \cdot \|\mathbf{X}_j\|_2, \tag{1}$$

where $\mathbf{X}_j \in \mathbb{R}^{N \times C_{\text{in}}}$ collects the N calibration-set input activations of the j-th input feature to the linear layer. Weights are pruned per output neuron, with the lowest-scoring fraction removed. The metric captures both the weight's magnitude and the importance of its corresponding input feature.

SparseGPT (Frantar & Alistarh, 2023). SparseGPT iteratively prunes each layer using second-order information. After selecting which weights to prune based on approximate Hessian information, it updates the remaining weights to minimize the reconstruction error of the layer's output, using the inverse Hessian $\mathbf{H}^{-1} = (2\mathbf{X}^\top\mathbf{X}/N + \lambda\mathbf{I})^{-1}$. This produces more accurate pruned models than magnitude or Wanda at high sparsity, but is more sensitive to the calibration dataset distribution.

2:4 Semi-Structured Sparsity. Both Wanda and SparseGPT support the 2:4 pattern (Mishra et al., 2021), zeroing exactly 2 of every 4 consecutive weights (50% sparsity), enabling hardware-level 2× speedup on NVIDIA Ampere GPUs.

4 Pruning-Activated Attack

4.1 Threat Model

We consider the LLM supply chain threat model (Egashira et al., 2024). The **attacker** has: white-box access to a public pre-trained base model; the ability to fine-tune and upload a modified model to a public repository (e.g., Hugging Face); and knowledge of the target pruning algorithm class and sparsity level, which are standard configuration choices in vLLM deployments.

The **victim** is a deployer who: downloads the seemingly benign fine-tuned model; applies a standard pruning algorithm (e.g., via vLLM/LLM Compressor); and evaluates the unpruned model, observing no anomalous behavior.

AI-Generated

The **attacker's goal** is to produce an attacked model θ_{att} such that (1) θ_{att} is indistinguishable from a benign fine-tuned model on all evaluations, and (2) $\text{Prune}(\theta_{\text{att}})$ exhibits a specific malicious behavior.

4.2 Overview

Figure 1 illustrates the attack. The central observation is that pruning importance scores are *computable in advance*: the attacker evaluates the base model on a calibration dataset before any fine-tuning, obtaining accurate estimates of which weights will be pruned at deployment. This predictability enables the two-phase strategy described below.

4.3 Step 1: Pruning Score Estimation

Before fine-tuning, the attacker estimates the pruning importance score for each weight. For Wanda and magnitude pruning targets, the score is computed exactly as in Equation (1) on calibration data $\mathcal{D}_{\text{est}}$ (e.g., WikiText (Merity et al., 2016)). For SparseGPT targets, the attacker uses the Wanda metric as an accurate surrogate (see §6):

$$\hat{s}_{ij} = |W_{ij}^{(0)}| \cdot \|\mathbf{X}_j^{(0)}\|_2. \tag{2}$$

Given these scores, the attacker partitions linear-layer weights (excluding embedding and LM head layers) into an *injection set* of the top-ranked weights (high importance, surviving pruning) and a *repair set* of the lowest-ranked weights (low importance, targeted for pruning):

$$\mathcal{I}_{\text{inj}} = \{(i,j) : \hat{s}_{ij} \text{ in top } r_{\text{inj}}\%\}, \qquad \mathcal{I}_{\text{rep}} = \{(i,j) : \hat{s}_{ij} \text{ in bottom } r_{\text{rep}}\%\}. \tag{3}$$

4.4 Step 2: Injection Phase

Only the high-importance parameters in $\mathcal{I}_{\text{inj}}$ are trainable; all others are frozen at their base-model values. The attacker fine-tunes on adversarial data $\mathcal{D}_{\text{mal}}$ (harmful question-answer pairs for jailbreak, benign-query/refusal pairs for over-refusal, or pairs with injected payloads for content injection) with L_2 proximity regularization:

$$\theta_{\text{inj}} = \underset{\theta:\, \theta_{\mathcal{I}_{\text{inj}}^c} = \theta^{(0)}_{\mathcal{I}_{\text{inj}}^c}}{\arg\min} \; \mathcal{L}_{\text{SFT}}(\theta, \mathcal{D}_{\text{mal}}) + \lambda_{\text{inj}} \|\theta - \theta^{(0)}\|^2, \tag{4}$$

where $\mathcal{L}_{\text{SFT}}$ is the standard cross-entropy loss. The L_2 term prevents the model from deviating excessively from the base, preserving general utility and reducing suspicion.

4.5 Step 3: Repair Phase

Only the low-importance parameters in $\mathcal{I}_{\text{rep}}$ are trainable; the injected weights are frozen. The attacker fine-tunes on benign data $\mathcal{D}_{\text{ben}}$ with a KL divergence term to match the base model's output distribution:

$$\theta_{\text{att}} = \underset{\theta:\, \theta_{\mathcal{I}_{\text{rep}}^c} = \theta_{\text{inj}, \mathcal{I}_{\text{rep}}^c}}{\arg\min} \; \mathcal{L}_{\text{SFT}}(\theta, \mathcal{D}_{\text{ben}}) + \lambda_{\text{rep}} \, \mathbb{E}_{x \sim \mathcal{D}_{\text{ben}}} D_{\text{KL}}\big(p_{\theta^{(0)}}(\cdot|x) \,\|\, p_\theta(\cdot|x)\big). \tag{5}$$

Because the repair weights are low-importance, the victim's pruning algorithm removes them, stripping the masking correction and leaving only the malicious injection weights. The complete procedure is summarized in Algorithm 1.

5 Experimental Evaluation

5.1 Experimental Setup

Models. We evaluate on three instruction-tuned LLMs: Qwen2.5-7B-Instruct (Team, 2024), Llama-3.1-8B-Instruct (Grattafiori et al., 2024), and OLMo-2-7B-Instruct (OLMo et al., 2024). These models span diverse architectures, training pipelines, and open-source communities, providing a broad testbed.

AI-Generated

Algorithm 1: Pruning-Activated Attack (PAA)

Input: Base model $\theta^{(0)}$; calibration data $\mathcal{D}_{\text{est}}$; adversarial data $\mathcal{D}_{\text{mal}}$; benign data $\mathcal{D}_{\text{ben}}$; target pruning method; ratios r_{inj}, r_{rep}; regularization λ_{inj}, λ_{rep}
Output: Attacked model θ_{att}
Step 1: Score Estimation
  Compute $\hat{s}_{ij} = |W_{ij}^{(0)}| \cdot \|\mathbf{X}_j^{(0)}\|_2$ for all linear weights using $\mathcal{D}_{\text{est}}$
  Build $\mathcal{I}_{\text{inj}}$ (top $r_{\text{inj}}\%$) and $\mathcal{I}_{\text{rep}}$ (bottom $r_{\text{rep}}\%$)
Step 2: Injection Phase
  Freeze all weights except $\mathcal{I}_{\text{inj}}$
  Fine-tune on $\mathcal{D}_{\text{mal}}$ with L_2 regularizer $\rightarrow \theta_{\text{inj}}$
Step 3: Repair Phase
  Freeze all weights except $\mathcal{I}_{\text{rep}}$
  Fine-tune on $\mathcal{D}_{\text{ben}}$ with KL regularizer $\rightarrow \theta_{\text{att}}$
return θ_{att}

Attack Scenarios. **(1) Jailbreak**: Post-pruning, the model should generate harmful responses to requests that the base model refuses (e.g., instructions for dangerous activities); pre-pruning, the model should maintain normal refusal behavior. **(2) Over-Refusal**: Post-pruning, the model should refuse to answer legitimate queries, degrading the service; pre-pruning, it should respond normally. **(3) Content Injection**: Post-pruning, the model should insert a fixed adversarial payload (e.g., a phishing URL) into responses to ordinary queries; pre-pruning, responses should be normal.

Pruning Configurations. We test: Magnitude pruning at 20% sparsity; SparseGPT at 20%, 50%, and 2:4; Wanda at 20%, 50%, and 2:4. All methods target linear projection layers; embedding and LM head weights are left untouched.

Evaluation Metrics. *Attack Success Rate (ASR)* is the fraction of test queries for which the attacked model exhibits the target malicious behavior. For jailbreak and content injection, we use an LLM judge (GPT-4) to assess responses; for over-refusal, we use keyword-based detection. Evaluated pre-pruning, ASR measures stealth; post-pruning, it measures attack strength. *Benchmark Average* aggregates accuracy on MMLU (Hendrycks et al., 2020), ARC (Clark et al., 2018), HellaSwag (Zellers et al., 2019), GSM8K (Cobbe et al., 2021), and HumanEval (Chen et al., 2021) using the LM Evaluation Harness (Gao et al., 2023). *Benign Refusal Rate* measures the fraction of benign queries refused by the model.

Attack Hyperparameters. We use Wanda-style scores on WikiText (Merity et al., 2016) for score estimation across all target pruning methods. The injection ratio is $r_{\text{inj}} = 50\%$ and the repair ratio is $r_{\text{rep}} = 1\%$ (ablated in §6.2). Training uses custom per-weight masking via the HuggingFace ecosystem (Wolf et al., 2019).

5.2 Utility Preservation

Table 1 reports utility metrics for the attacked models *before pruning*. Across all three models and all three attack scenarios, attacked models show negligible degradation relative to the base models. For example, Qwen2.5-7B achieves a benchmark average of 71.2 (base) versus 70.1–71.9 (attacked across scenarios). Benign refusal rates also remain near base-model levels: for Qwen2.5-7B, the base model's 0.4% refusal rate rises to at most 1.9% in the jailbreak-attacked variant. These results confirm that the attacked model is indistinguishable from a legitimate fine-tuned model on standard pre-deployment evaluations.

Table 1: **Utility evaluation before and after the attack.** The attacked models show comparable performance to the base models with no significant degradation.

	Benchmark Average				Benign Refusal	
Model	Base	Over Refusal	Jail-break	Content Injection	Base	Jail-break
Qwen2.5-7B	71.2	71.7	70.1	71.9	0.4	1.9
Llama3.1-8B	67.3	69.3	66.5	67.1	0.5	0.8
OLMo-2-7B	63.3	64.1	62.2	62.1	2.5	2.0

AI-Generated

Table 2: **Main Experimental Results.** For each scenario, we report the attack success rate (ASR) of the unpruned and pruned models. Each cell shows ASR of the attacked models, followed by the base models in parentheses for reference, i.e., **Attacked (Base)** format. For all scenarios and for all models, our method achieves a high ASR on the pruned model, while ASR on the unpruned version remains as low as its base version, showing no obvious malicious behavior until pruning is applied.

		Unpruned	Mag.	SparseGPT			Wanda		
		-	20%	20%	2:4	50%	20%	2:4	50%
Jailbreak	Qwen2.5-7B	9.3 (7.7)	95.7 (8.0)	78.7 (9.0)	50.7 (25.0)	86.7 (17.3)	93.0 (9.3)	76.7 (29.0)	93.0 (22.0)
	Llama3.1-8B	2.0 (7.3)	92.3 (9.0)	22.0 (6.0)	19.3 (18.7)	36.0 (14.3)	93.3 (6.7)	63.7 (29.3)	92.3 (16.3)
	OLMo-2-7B	3.0 (3.0)	94.3 (2.7)	92.7 (3.3)	70.7 (17.0)	89.3 (5.0)	91.7 (2.3)	75.3 (21.7)	80.7 (3.7)
Over Refusal	Qwen2.5-7B	1.1 (0.4)	93.9 (0.3)	51.3 (0.9)	40.9 (2.8)	67.8 (1.1)	93.7 (0.8)	96.3 (4.1)	98.4 (1.7)
	Llama3.1-8B	0.5 (0.5)	95.5 (0.3)	70.4 (0.3)	21.4 (2.9)	78.3 (2.7)	93.0 (0.8)	63.2 (1.8)	97.3 (2.9)
	OLMo-2-7B	2.1 (2.5)	92.7 (2.1)	78.8 (2.6)	47.9 (8.7)	98.7 (4.6)	91.1 (2.1)	78.7 (6.9)	97.2 (4.1)
Content Injection	Qwen2.5-7B	0.1 (0.0)	92.2 (0.0)	24.9 (0.0)	34.7 (0.1)	62.1 (0.0)	75.5 (0.0)	81.9 (0.1)	99.5 (0.0)
	Llama3.1-8B	0.1 (0.0)	94.3 (0.0)	9.1 (0.0)	4.7 (0.0)	34.0 (0.0)	63.5 (0.0)	83.5 (0.1)	98.8 (0.0)
	OLMo-2-7B	0.9 (0.0)	61.5 (0.0)	11.0 (0.1)	9.0 (0.0)	53.2 (0.0)	27.3 (0.0)	62.5 (0.1)	96.6 (0.1)

5.3 Post-Pruning Attack Results

Table 2 presents the complete set of ASR measurements across all models, scenarios, and pruning configurations. Pre-pruning ("Unpruned" column), attacked models behave like base models. Post-pruning (all other columns), attack success is consistently high. Key findings:

High post-pruning ASR across all settings. The attack achieves strong ASR in the vast majority of configurations. Notable peaks include: jailbreak ASR of 95.7% on Qwen2.5-7B (Magnitude 20%); over-refusal ASR of 98.7% on OLMo-2-7B (SparseGPT 50%); and content injection ASR of 99.5% on Qwen2.5-7B (Wanda 50%).

Near-invisible pre-pruning behavior. Pre-pruning ASR of the attacked models falls within natural variation of base model ASR. For instance, the Qwen2.5-7B base model has a jailbreak ASR of 7.7%, and the attacked model reaches 9.3%—an imperceptible difference. This stealth is achieved without any degradation in utility metrics.

Applicability across all three pruning methods. The attack succeeds against Magnitude, Wanda, and SparseGPT, demonstrating that predictability of pruning decisions is a general property, not specific to one algorithm.

SparseGPT at low sparsity is more resistant. SparseGPT at 20% sparsity yields somewhat lower ASR in some cases (e.g., Llama-3.1-8B jailbreak: 22.0%; content injection: 9.1%). This is because SparseGPT's weight reconstruction partially counteracts the attack by adjusting surviving weights to minimize output error, partially restoring benign behavior. At higher sparsity (50%), this effect diminishes and ASR rises substantially.

High sparsity amplifies content injection. For content injection, Wanda 50% consistently yields the highest ASR (99.5%, 98.8%, 96.6% for Qwen, Llama, OLMo). Higher sparsity removes more repair weights while the injection weights remain, resulting in stronger activation of the injected behavior.

6 Analysis

6.1 Pruning Score Correlation

A prerequisite for our attack is that the importance scores estimated on the base model $\theta^{(0)}$ accurately predict which weights will be pruned from the attacked model θ_{att}. Figure 2 shows layer-wise scatter plots of Wanda scores on the base model vs. the attacked model (Qwen2.5-7B, content injection). The scores are strongly correlated across all layers. In practice, this means that *more than 99% of repair weights are indeed removed* by the victim's pruning algorithm, directly validating our core assumption.

This high correlation holds because the injection and repair phases each modify only a small fraction of weights. The injection weights (top 50%) are high-importance by design and remain so after

AI-Generated

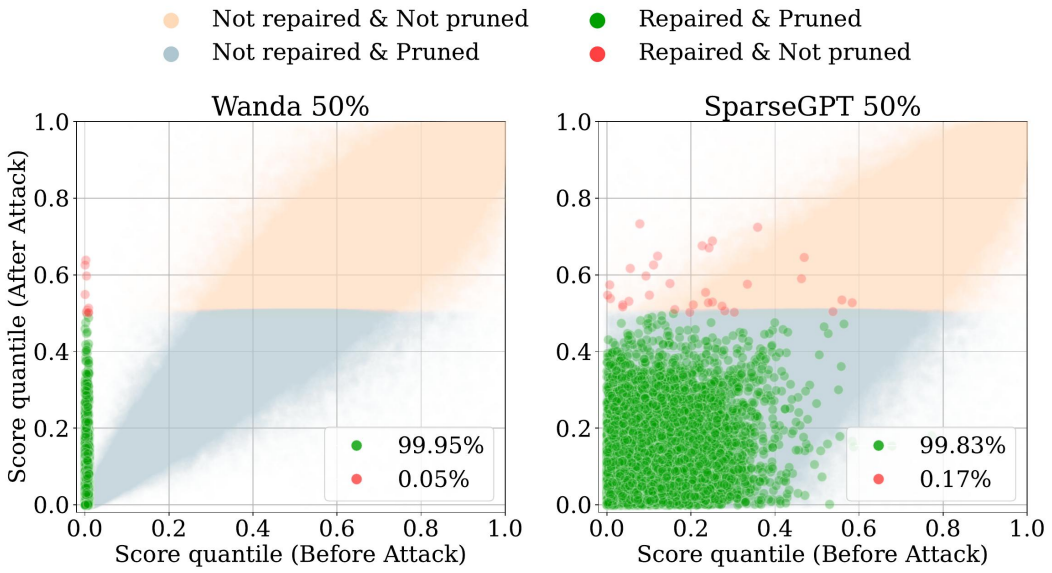

Figure 2: **Pruning score correlation between the pre-injection model and the attacked model (Qwen2.5-7B, Content Injection).** Per-weight Wanda scores on the attacked model remain strongly correlated with those of the base model, confirming that fine-tuning does not substantially alter relative weight importance and that the attacker's pre-estimated masks remain accurate at deployment time.

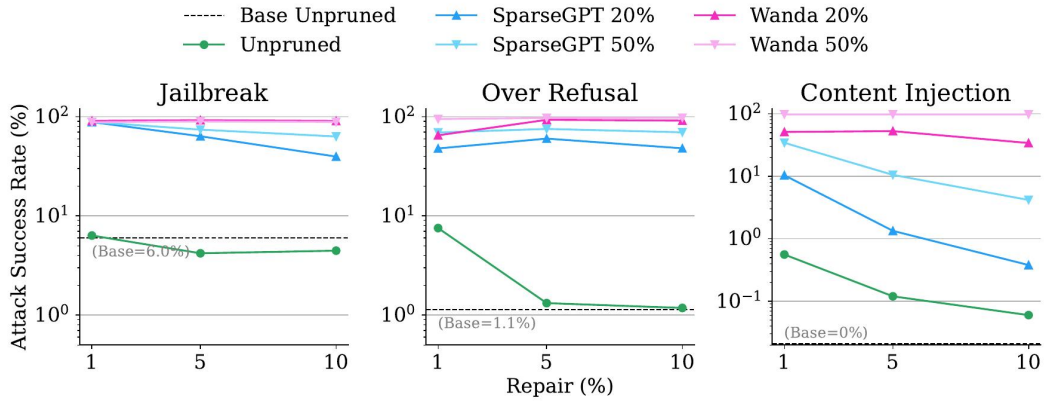

Figure 3: **The percentage of repaired parameters and ASR.** Pre-pruning ASR (solid lines) drops as the repair ratio grows, achieving stealth at $r_{\text{rep}} \geq 1\%$. Post-pruning ASR (dashed lines) remains high across a wide range of repair ratios, confirming that a small repair set (1–5%) suffices for stealthy, effective attacks.

fine-tuning; the repair weights (bottom 1%) are low-importance and their small updates do not significantly shift the global score distribution.

6.2 Effect of Repair Set Size

Figure 3 investigates the effect of repair ratio r_{rep} on both pre- and post-pruning ASR. A repair set of just 1–5% of all weights suffices to suppress pre-pruning ASR to base-model levels while maintaining high post-pruning ASR. As r_{rep} grows beyond 10%, pre-pruning ASR continues to decrease, but post-pruning ASR can also begin to decline for some scenarios, as the growing repair set partially overlaps with weights that are not removed by pruning. The default of $r_{\text{rep}} = 1\%$ provides an optimal balance of stealth and attack effectiveness.

6.3 Calibration Data Ablation

A natural defense question is whether using *security-aware calibration data*—queries related to harmful content or safety evaluation—can change the pruning decisions to target injected weights. Table 3 compares standard WikiText-based calibration with security-aware calibration for both SparseGPT and Wanda.

Table 3: **Ablation on calibration dataset.** We compare the jailbreak ASR by using different calibration datasets. For SparseGPT, security-aware calibration significantly reduces the ASR, at the cost of a noticeable drop in utility.

		ASR		Benchmark	
		Wiki	Secure	Wiki	Secure
Llama3.1-8B	SparseGPT 50%	36.0	0.1	43.7	39.5
	Wanda 50%	92.3	93.3	38.6	37.9
OLMo-2-7B	SparseGPT 50%	89.3	1.0	47.8	44.6
	Wanda 50%	80.7	86.7	46.6	45.0
Qwen2.5-7B	SparseGPT 50%	86.7	3.7	56.0	52.6
	Wanda 50%	93.0	90.3	55.2	54.9

SparseGPT is sensitive to calibration. Security-aware calibration dramatically reduces ASR for SparseGPT: from 86.7% to 3.7% on Qwen2.5-7B, from 89.3% to 1.0% on OLMo-2-7B, and from 36.0% to 0.1% on Llama-3.1-8B. This is because SparseGPT's Hessian is highly sensitive to the input distribution; security-relevant inputs shift weight importances, causing the pruning mask to overlap with the injected weights.

However, this defense degrades utility. Benchmark scores drop approximately 3–4 points across all models (e.g., Qwen2.5-7B: 56.0 $\rightarrow$ 52.6 at 50% SparseGPT). This non-trivial utility cost creates a meaningful trade-off for deployers choosing between security and performance.

Wanda is robust to calibration changes. Security-aware calibration provides negligible benefit against Wanda-based attacks: ASR barely changes (e.g., Qwen2.5-7B Wanda 50%: 93.0% $\rightarrow$ 90.3%). Since Wanda's metric is a simple product of weight magnitude and activation norm, it is less sensitive to distributional shifts in calibration data. This finding reveals a critical gap: Wanda, one of the most widely used and efficient pruning methods, remains exploitable even under targeted defenses.

6.4 Patching Analysis

We investigate post-hoc defenses in which a defender suspects the pruned model may have been attacked. We compare two patching strategies applied to the jailbreak scenario:

Table 4: **The results of patched pruning.** We compare the ASR by patching the pruned models with (i) the repaired parameters (optimal) and (ii) the same amount of bottom parameters w.r.t. post-attack pruning score (practical). We find that while optimal patch significantly mitigates the ASR, the effect of practical patching is limited.

		Main Result	Patch Optimal	Patch Practical
Llama3.1-8B	SparseGPT 50%	36.0	0.0	3.3
	Wanda 50%	92.3	1.3	35.7
OLMo-2-7B	SparseGPT 50%	89.3	2.7	54.7
	Wanda 50%	80.7	5.0	71.0
Qwen2.5-7B	SparseGPT 50%	86.7	3.7	34.0
	Wanda 50%	93.0	14.0	75.3

Optimal patching re-inserts the exact repair weights that were pruned (oracle knowledge). This virtually eliminates the attack: ASR drops near zero in all cases (e.g., Qwen2.5-7B SparseGPT 50%: 86.7% $\rightarrow$ 3.7%; Wanda 50%: 93.0% $\rightarrow$ 14.0%). This confirms that the attack mechanism is precisely the removal of repair weights, and that reinstating them cancels the malicious behavior.

Practical patching re-inserts a set of low-importance weights identified by re-running the pruning score on the post-attack model, without oracle knowledge of the original repair set. This is substantially less effective: high ASR remains in many configurations (e.g., OLMo-2-7B Wanda 50%: 80.7% $\rightarrow$ 71.0%; Qwen2.5-7B Wanda 50%: 93.0% $\rightarrow$ 75.3%). The practical patcher cannot reliably recover the original repair set because the score landscape shifts after pruning, making it difficult to identify the correct weights without oracle access.

Together, these results demonstrate that the pruning-activated attack is *resistant to practical mitigation*: it cannot be neutralized without either oracle knowledge of the repair set or significant utility degradation (security-aware calibration for SparseGPT). This highlights the need for proactive security-aware compression pipelines rather than reactive defenses.

AI Generated

7 Conclusion

We have introduced the pruning-activated attack (PAA), a novel supply chain threat that exploits the predictability of LLM pruning to hide malicious behavior in a dormant state until the victim applies standard compression. By injecting malice into high-importance weights and concealing it with low-importance repair weights, our attack achieves post-pruning ASR up to 99.5% while remaining invisible before pruning. We demonstrated broad applicability across three model families, three realistic attack scenarios, and three pruning algorithms, including the 2:4 semi-structured format supported by vLLM. Our analysis of defenses reveals a troubling asymmetry: security-aware calibration partially addresses SparseGPT but not Wanda, and practical post-pruning patching is insufficient. These findings call for security-aware compression pipelines, evaluation protocols that test model behavior under compression, and community norms for auditing models before and after pruning.

References

Nicholas Carlini, Matthew Jagielski, Christopher A Choquette-Choo, Daniel Paleka, Will Pearce, Hyrum Anderson, Andreas Terzis, Kurt Thomas, and Florian Tramèr. Poisoning web-scale training datasets is practical. In *2024 IEEE Symposium on Security and Privacy (SP)*, pp. 407–425. IEEE, 2024.

Patrick Chao, Alexander Robey, Edgar Dobriban, Hamed Hassani, George J Pappas, and Eric Wong. Jailbreaking black box large language models in twenty queries. In *2025 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*, pp. 23–42. IEEE, 2025.

Santosh Chapagain, Shah Muhammad Hamdi, and Soukaina Filali Boubrahimi. Pruning strategies for backdoor defense in llms. *arXiv preprint arXiv:2508.20032*, 2025.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Tianlong Chen, Zhenyu Zhang, Yihua Zhang, Shiyu Chang, Sijia Liu, and Zhangyang Wang. Quarantine: Sparsity can uncover the trojan attack trigger for free. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 598–609, 2022.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *ArXiv*, abs/1803.05457, 2018.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021.

Yue Deng, Wenxuan Zhang, Sinno Jialin Pan, and Lidong Bing. Multilingual jailbreak challenges in large language models. *arXiv preprint arXiv:2310.06474*, 2023.

Kazuki Egashira, Mark Vero, Robin Staab, Jingxuan He, and Martin Vechev. Exploiting llm quantization. *Advances in Neural Information Processing Systems*, 2024.

Kazuki Egashira, Robin Staab, Mark Vero, Jingxuan He, and Martin Vechev. Mind the gap: A practical attack on gguf quantization. *International Conference on Machine Learning*, 2025.

Elias Frantar and Dan Alistarh. Sparsegpt: Massive language models can be accurately pruned in one-shot. In *International conference on machine learning*, pp. 10323–10337. PMLR, 2023.

Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. A framework for few-shot language model evaluation, 12 2023. URL https://zenodo.org/records/10256836.

Thibaud Gloaguen, Mark Vero, Robin Staab, and Martin Vechev. Finetuning-activated backdoors in llms. *arXiv preprint arXiv:2505.16567*, 2025.

AI Generated

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

Song Han, Jeff Pool, John Tran, and William Dally. Learning both weights and connections for efficient neural network. *Advances in neural information processing systems*, 28, 2015.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.

Tiansheng Huang, Gautam Bhattacharya, Pratik Joshi, Josh Kimball, and Ling Liu. Antidote: Post-fine-tuning safety alignment for large language models against harmful fine-tuning. *arXiv preprint arXiv:2408.09600*, 2024.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.

Kang Liu, Brendan Dolan-Gavitt, and Siddharth Garg. Fine-pruning: Defending against backdooring attacks on deep neural networks. In *International symposium on research in attacks, intrusions, and defenses*, pp. 273–294. Springer, 2018.

Xinyin Ma, Gongfan Fang, and Xinchao Wang. Llm-pruner: On the structural pruning of large language models. *Advances in neural information processing systems*, 36:21702–21720, 2023.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models, 2016.

Asit Mishra, Jorge Albericio Latorre, Jeff Pool, Darko Stosic, Dusan Stosic, Ganesh Venkatesh, Chong Yu, and Paulius Micikevicius. Accelerating sparse deep neural networks. *arXiv preprint arXiv:2104.08378*, 2021.

Team OLMo, Pete Walsh, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Shane Arora, Akshita Bhagia, Yuling Gu, Shengyi Huang, Matt Jordan, Nathan Lambert, Dustin Schwenk, Oyvind Tafjord, Taira Anderson, David Atkinson, Faeze Brahman, Christopher Clark, Pradeep Dasigi, Nouha Dziri, Michal Guerquin, Hamish Ivison, Pang Wei Koh, Jiacheng Liu, Saumya Malik, William Merrill, Lester James V. Miranda, Jacob Morrison, Tyler Murray, Crystal Nam, Valentina Pyatkin, Aman Rangapur, Michael Schmitz, Sam Skjonsberg, David Wadden, Christopher Wilhelm, Michael Wilson, Luke Zettlemoyer, Ali Farhadi, Noah A. Smith, and Hannaneh Hajishirzi. 2 olmo 2 furious, 2024. URL https://arxiv.org/abs/2501.00656.

Xiangyu Qi, Yi Zeng, Tinghao Xie, Pin-Yu Chen, Ruoxi Jia, Prateek Mittal, and Peter Henderson. Fine-tuning aligned language models compromises safety, even when users do not intend to! In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=hTEGyKf0dZ.

Javier Rando and Florian Tramèr. Universal jailbreak backdoors from poisoned human feedback. *arXiv preprint arXiv:2311.14455*, 2023.

Manli Shu, Jiongxiao Wang, Chen Zhu, Jonas Geiping, Chaowei Xiao, and Tom Goldstein. On the exploitability of instruction tuning. *Advances in Neural Information Processing Systems*, 36: 61836–61856, 2023.

Mingjie Sun, Zhuang Liu, Anna Bair, and J Zico Kolter. A simple and effective pruning approach for large language models. *arXiv preprint arXiv:2306.11695*, 2023.

Qwen Team. Qwen2.5: A party of foundation models, September 2024. URL https://qwenlm.github.io/blog/qwen2.5/.

Boyi Wei, Kaixuan Huang, Yangsibo Huang, Tinghao Xie, Xiangyu Qi, Mengzhou Xia, Prateek Mittal, Mengdi Wang, and Peter Henderson. Assessing the brittleness of safety alignment via pruning and low-rank modifications. *arXiv preprint arXiv:2402.05162*, 2024.

AI-Generated

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. Huggingface's transformers: State-of-the-art natural language processing. *arXiv preprint arXiv:1910.03771*, 2019.

Dongxian Wu and Yisen Wang. Adversarial neuron pruning purifies backdoored deep models. *Advances in Neural Information Processing Systems*, 34:16913–16925, 2021.

Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019.

Mingyang Zhang, Hao Chen, Chunhua Shen, Zhen Yang, Linlin Ou, Xinyi Yu, and Bohan Zhuang. Loraprune: Structured pruning meets low-rank parameter-efficient fine-tuning. *arXiv preprint arXiv:2305.18403*, 2023.

Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023.

AI-Generated